

Data Fetching Strategies

What is Cache · Where Cache Lives · Lambda + Vercel Architecture

fetch() Options · Parallel Fetch · Revalidation · Streaming · SWR

Akshay Chavhan | Target: Syngenta India + All Companies | April 2026

■ BEGINNER LEVEL

What is Cache? (From Scratch)

Cache means **saving an answer so you do not ask the same question again**. In web terms: save an API response on the server so the API is not called on every user visit.

WITHOUT CACHE:

1000 users visit /products

Each user → Lambda wakes up → calls API (500ms) → Lambda dies

= 1000 API calls, 1000 Lambda executions (slow + expensive)

WITH CACHE (saved on SERVER, not in browser):

User 1 → Lambda calls API → saves to Vercel Data Cache

Users 2-1000 → Lambda reads from Vercel Data Cache instantly

= 1 API call + 999 cache hits (fast + cheap)

KEY POINT: Cache is on the SERVER — all users benefit from it.

Browser cache is personal — only helps that same person.

■ Zomato Analogy: First search for pizza takes 2s. Second search in 30s = instant (cached). Next day = fresh search again. This is exactly ISR — cache for N seconds then refresh.

Where Can You Fetch Data?

Location	How	Best For
Server Components (default)	async/await directly	DB, APIs, most data — no boilerplate
Route Handlers (api/route.ts)	GET/POST/PUT handler	External APIs, webhooks
Server Actions	Form submit, mutations	Create, update, delete operations
Client Components	useEffect + SWR / React Query	User-specific, real-time, post-auth

Server Component Fetch — No Boilerplate

	React (old way)	Next.js Server Component
State needed	useState([])	No

Loading state	useState(true)	No
Side effect	useEffect(()=>{...},[])	No
Data fetch	fetch inside useEffect	await fetch() directly
Where runs	Browser (CSR)	Server
SEO	Poor (blank HTML)	Excellent (pre-rendered)

```
// Next.js Server Component – clean and simple
export default async function ProductsPage() {
  const res = await fetch("https://api.example.com/products");
  const products = await res.json();
  return <ul>{products.map(p => <li key={p.id}>{p.name}</li>)}</ul>;
}
// No useState. No useEffect. No loading state. Just async/await. ■
```

The 5 fetch() Cache Options

Next.js **extends** the native fetch() with extra options. These tell the Next.js server how long to cache the response. The next:{} key is what Next.js added — it does not exist in standard browser fetch.

```
// 1. SSG – cache at build time, never refresh (default)
await fetch("https://api.example.com/data");

// 2. SSG explicit – same as above
await fetch("https://api.example.com/data", { cache: "force-cache" });

// 3. SSR – never cache, hit API on every user request
await fetch("https://api.example.com/data", { cache: "no-store" });

// 4. ISR – cache for 3600 seconds, then auto-refresh in background
await fetch("https://api.example.com/data", { next: { revalidate: 3600 } });
//                               ↑ This is what Next.js added

// 5. On-demand ISR – cache until you manually clear it
await fetch("https://api.example.com/data", { next: { tags: ["products"] } });
//                               ↑ This too is added by Next.js
```

Option	Strategy	Re-fetches When	Syngenta Use Case
(no option)	SSG	Never (until redeploy)	About page, Terms
cache:"force-cache"	SSG	Same — explicit	Documentation pages
cache:"no-store"	SSR	Every request	Farmer portfolio, login
next:{revalidate:N}	ISR	Every N seconds	Crop prices, news
next:{tags:["x"]}	On-demand ISR	revalidateTag("x") called	Admin-managed content

Syngenta Real Example — Which Option for What?

```
// 1. About Syngenta page – never changes
const res = await fetch("https://api.syngenta.com/about");
// SSG – cached at build, served from CDN forever

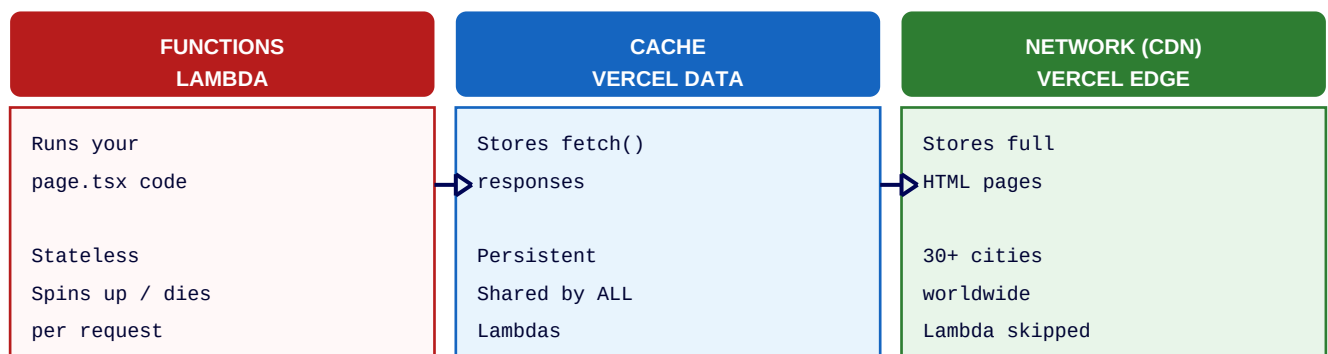
// 2. Crop prices – changes every hour
const res = await fetch("https://api.syngenta.com/crop-prices", {
  next: { revalidate: 3600 } // ISR – refresh every 1 hour
});

// 3. Farmer personal portfolio – different per user
const res = await fetch("https://api.syngenta.com/my-portfolio", {
  cache: "no-store" // SSR – always fresh, never cache
});
```

■ INTERMEDIATE LEVEL

Where Cache Actually Lives — Vercel Architecture

Since Next.js on Vercel = Lambda functions (stateless — they die after each request), cache CANNOT live inside the Lambda. Vercel has **separate persistent layers**:



```
Lambda A: fetch(url) → calls API → saves to Vercel Data Cache → Lambda dies
Lambda B: fetch(url) → checks Vercel Data Cache → HIT! → instant result ■
Lambda C: fetch(url) → checks Vercel Data Cache → HIT! → instant result ■
```

All Lambdas share ONE Vercel Data Cache (like chefs sharing one fridge)
Cache survives Lambda death – it lives separately on Vercel infrastructure

For SSG pages: HTML saved to Edge CDN → Lambda not even called! ■

■ On your OWN server (not Vercel): cache saves to .next/cache/ folder. With multiple servers, use Redis/Upstash as a shared cache handler so all servers read from the same place.

How to SEE Cache Working

```
// METHOD 1: Browser DevTools → Network Tab → Response Headers
x-nextjs-cache: HIT      ← served from cache (fast!) ■
x-nextjs-cache: MISS     ← not in cache, fetched fresh
x-nextjs-cache: STALE    ← from cache but revalidating in background
x-nextjs-cache: SKIP     ← cache skipped (SSR / no-store)

// METHOD 2: Console timing – shows real vs cached speed
const start = Date.now();
const res = await fetch("https://api.example.com/products", {
  next: { revalidate: 30 }
});
console.log(`Took: ${Date.now() - start}ms`);
// First visit → "Took: 387ms" (real API call – MISS)
// Second visit → "Took: 2ms" (from cache – HIT) ■
// After 30s → "Took: 412ms" (cache expired – MISS again)

// METHOD 3: File system – .next/cache/fetch-cache/
// Each cached response = one hashed file containing URL + data + tags
```

Parallel vs Sequential Fetching

Sequential creates a **waterfall** — each request waits for previous. Parallel with **Promise.all()** fires all requests simultaneously. This is the most common performance mistake in Next.js apps.

```
// ■ SEQUENTIAL – waterfall (slow!)
async function SlowDashboard() {
  const user = await fetchUser(); // 200ms – starts, waits
  const crops = await fetchCrops(); // 300ms – waits after user done
  const weather = await fetchWeather(); // 250ms – waits after crops done
  // Total: 200 + 300 + 250 = 750ms ■
}

// ■ PARALLEL – all at same time (fast!)
async function FastDashboard() {
  const [user, crops, weather] = await Promise.all([
    fetchUser(), // 200ms ■■
    fetchCrops(), // 300ms ■■■ all start simultaneously
    fetchWeather(), // 250ms ■■
  ]);
  // Total: 300ms (only slowest one matters) ■
}
```

fetch() Memoization — Auto Deduplication

Next.js auto-deduplicates **identical** fetch calls in one render pass. Same URL called in layout.tsx AND page.tsx = only ONE real network request.

```
// layout.tsx — needs user for navbar
async function DashboardLayout({ children }) {
  const user = await fetch("/api/user").then(r=>r.json()); // Request fires
  return <><Navbar user={user}/>{children}</>;
}

// page.tsx — also needs user
async function DashboardPage() {
  const user = await fetch("/api/user").then(r=>r.json()); // DEDUPLICATED ■
  return <h1>Welcome {user.name}</h1>;
}
// Result: Only 1 network request. Both get same data. Automatic.
// Works: same URL, same options, GET requests, same render pass only
```

Fetching with Prisma/DB — Route Segment Config

Prisma queries are not `fetch()` calls — cache options do not apply. Use Route Segment Config to control the whole page caching strategy.

```
// app/dashboard/page.tsx

// Option A: ISR — cache the DB result, refresh every hour
export const revalidate = 3600;
export default async function Page() {
  const crops = await db.crop.findMany(); // Prisma — cached via revalidate
  return <table>...</table>;
}

// Option B: SSR — always fresh, hit DB on every request
export const dynamic = "force-dynamic";
export default async function Page() {
  const user = await db.user.findUnique({ where: { id: getUser() } });
  return <div>Hello {user?.name}</div>;
}

// Quick reference:
// export const revalidate = false      → SSG (cache forever)
// export const revalidate = 0          → SSR (never cache)
// export const revalidate = 3600       → ISR (every 1 hour)
// export const dynamic = "force-dynamic" → always SSR
// export const dynamic = "force-static" → always SSG
```

On-Demand Revalidation — `revalidateTag` and `revalidatePath`

Instead of waiting for a timer, **you decide exactly when to clear cache**. Perfect for admin-managed content — clear cache instantly when data changes.

```
// STEP 1: Tag your fetch
async function getProducts() {
  const res = await fetch("https://api.example.com/products", {
    next: { tags: ["products"] } // give this cache entry a name
  });
  return res.json();
}

// STEP 2: Clear that tag when data changes (Server Action)
"use server";
import { revalidateTag } from "next/cache";

export async function addProduct(formData: FormData) {
  await db.product.create({ data: { name: formData.get("name") as string } });
  revalidateTag("products"); // ← instantly clears ALL "products" caches ■
}

// PATH-BASED — clear a specific URL
import { revalidatePath } from "next/cache";
export async function updatePrice(cropId: string, price: number) {
  await db.crop.update({ where: { id: cropId }, data: { price } });
  revalidatePath("/products"); // clears /products page
  revalidatePath(`/products/${cropId}`); // clears this product page
}
```

	revalidateTag("products")	revalidatePath("/products")
What it clears	ALL fetches tagged "products" anywhere	All data on /products URL only
Scope	Multiple pages at once	One specific page
Use when	Same data on many pages	You know exact page to refresh
Example	Product list on home + shop + search	Just the /products listing page

Client-Side Fetching — useEffect vs SWR

Use client fetching only for: user-specific data after auth, real-time updates.

```
// useEffect — manual, verbose (avoid if possible)
"use client";
import { useEffect, useState } from "react";

function UserOrders() {
  const [orders, setOrders] = useState(null);
  const [loading, setLoading] = useState(true);
  useEffect(() => {
    fetch("/api/my-orders").then(r=>r.json())
      .then(d => { setOrders(d); setLoading(false); });
  }, []);
  if (loading) return <p>Loading...</p>;
  return <ul>{orders.map(o => <li>{o.name}</li>)}</ul>;
}

// SWR — recommended (auto cache, retry, revalidation)
"use client";
import useSWR from "swr";
const fetcher = (url: string) => fetch(url).then(r => r.json());

function UserOrders() {
  const { data, error, isLoading } = useSWR("/api/my-orders", fetcher);
  if (isLoading) return <p>Loading...</p>;
  if (error) return <p>Error!</p>;
  return <ul>{data.map(o => <li>{o.name}</li>)}</ul>;
}
// SWR: auto-retries, refetches on window focus, caches on client too
```

■ EXPERT LEVEL

The 4 Caching Layers — Complete Architecture

1. Request Memoization	Inside Lambda RAM Per render pass only Deduplicates fetch() calls
2. Data Cache (Server)	Vercel Data Cache Persistent, shared by ALL Lambdas Controlled by revalidate
3. Full Route Cache (CDN)	Vercel Edge Network Complete HTML pages Lambda not called for cache hits
4. Router Cache (Browser)	User browser RAM RSC payloads for visited pages 30s to 5 minutes

Layer	Cleared By	Who Benefits
1. Request Memoize	Automatic — Lambda dies	Same component tree, one render
2. Data Cache	revalidateTag / revalidatePath	ALL users — shared on server
3. Full Route Cache	revalidateTag / revalidatePath	ALL users — from CDN globally
4. Router Cache	router.refresh() / page reload	That user only — browser RAM

unstable_cache — Cache Prisma/DB Queries Like fetch()

fetch() has built-in caching. Prisma/DB queries do NOT. unstable_cache wraps ANY async function with full Next.js caching — tags, revalidation, everything.

```

import { unstable_cache } from "next/cache";
import { db } from "@/lib/db";

// Wrap Prisma query with cache + revalidation + tags
export const getCachedCropStats = unstable_cache(
  async (region: string) => {
    return db.crop.groupBy({ // Prisma – now fully cached ■
      by: ["type"],
      where: { region },
      _avg: { price: true },
    });
  },
  ["crop-stats"], // cache key prefix
  {
    tags: ["crop-stats"], // for revalidateTag()
    revalidate: 3600, // refresh every hour
  }
);

// Usage in Server Component
export default async function StatsPage() {
  const stats = await getCachedCropStats("punjab");
  return <div>Wheat avg: Rs.{stats[0]._avg.price}/kg</div>;
}

// Clear it on demand
import { revalidateTag } from "next/cache";
export async function updateCropData() {
  await db.crop.updateMany({ ... });
  revalidateTag("crop-stats"); // clears getCachedCropStats cache
}

```

Streaming with Suspense — Progressive Data Loading

Without Suspense: server waits for ALL data → one big response → user waits. With Suspense: each section streams in independently as its data arrives. Users see content progressively — fast sections appear instantly.


```
// app/dashboard/page.tsx
import { Suspense } from "react";

export default function DashboardPage() {
  return (
    <div>
      <h1>Syngenta Dashboard</h1> { /* Renders instantly */}

      <Suspense fallback={<p>Loading prices...</p>}>
        <CropPricesTable /> { /* Streams when Prisma query done */}
      </Suspense>

      { /* Slow weather API does NOT block crop prices above */}
      <Suspense fallback={<p>Loading weather...</p>}>
        <WeatherWidget /> { /* Streams independently */}
      </Suspense>
    </div>
  );
}

async function CropPricesTable() {
  const crops = await db.crop.findMany(); // only blocks THIS section
  return <table>...</table>;
}

async function WeatherWidget() {
  const w = await fetch("https://api.weather.com", { cache: "no-store" })
    .then(r=>r.json());
  return <div>Temp: {w.temp}C</div>;
}
```

■ Critical Gotchas

■ Gotcha 1: Lowest revalidate wins for the ENTIRE page. Two fetches with revalidate:3600 and revalidate:60 — the whole page revalidates every 60 seconds. Fix: split into separate Suspense boundaries so each has independent revalidation.

■ Gotcha 2: Using cookies(), headers(), or searchParams() in a Server Component makes the ENTIRE route SSR — even fetches with force-cache above that line become dynamic. Move dynamic APIs to leaf components.

■ Gotcha 3: fetch() memoization is per-user per-request. User A and User B each get separate memoization. Only deduplicates within ONE render pass for ONE user.

■ Gotcha 4: revalidateTag() clears server cache (layers 2+3) but NOT the browser Router Cache (layer 4). User may still see stale data for ~30s. Fix: call router.refresh() on the client after Server Action completes.

■ Best Practice: Always use Promise.all() for independent fetches. Never await sequentially when fetches do not depend on each other.

■ Best Practice: Wrap every Prisma/DB query that benefits from caching in unstable_cache with a tag. Without it every request hits the database. Add revalidateTag() call wherever that data is mutated.

■ INTERVIEW Q&A WITH DETAILED ANSWERS

Beginner Level

BEGINNER

Q1. What is cache and why does Next.js use it?

Answer: Cache means saving a response so you do not fetch it again. Without cache: 1000 users = 1000 API calls. With cache: 1000 users = 1 API call + 999 served from cache. Cache is stored on the SERVER — not in the browser — so all users benefit. Next.js uses it to make pages fast and reduce API/DB costs.

BEGINNER

Q2. What are the `fetch()` cache options in Next.js?

Answer: No option = SSG (cached at build forever). `cache:"force-cache"` = same, explicit. `cache:"no-store"` = SSR (fresh every request). `next:{revalidate:N}` = ISR (cached N seconds then auto-refresh). `next:{tags:["x"]}` = ISR with tag for on-demand invalidation via `revalidateTag`.

BEGINNER

Q3. How is data fetching different in Next.js vs React?

Answer: React uses `useEffect` + `useState` in Client Components — blank screen, loading state, JS must download first. Next.js Server Components are async — just await data directly. No hooks, no loading state, pre-rendered HTML sent immediately. Better SEO, faster first paint, simpler code.

Intermediate Level

INTERMEDIATE

Q4. Next.js uses Lambda functions on Vercel. Lambda is stateless and dies. So where is cache saved?

Answer: Cache is NOT in the Lambda. Vercel has a separate persistent Data Cache (key-value store) that all Lambda instances share. Lambda A fetches, writes to Vercel Data Cache, dies. Lambda B reads from Vercel Data Cache — instant, no API call. For SSG pages, full HTML is pushed to Vercel Edge Network (CDN across 30+ cities) — Lambda is not even called for cached pages.

INTERMEDIATE

Q5. What is the difference between sequential and parallel fetching?

Answer: Sequential: each `await` waits for previous — total time = sum of all (waterfall). Parallel: `Promise.all([...])` fires all simultaneously — total time = only slowest. Example: 3 fetches of 200ms, 300ms, 250ms. Sequential = 750ms. Parallel = 300ms. Always use `Promise.all` for independent fetches.

INTERMEDIATE**Q6. What is fetch memoization in Next.js?**

Answer: Auto-deduplication of identical `fetch()` calls within the same render pass. If `layout.tsx` and `page.tsx` both call `fetch("/api/user")`, only ONE real network request fires. Works for GET requests only, within one render pass, not across users or requests.

INTERMEDIATE**Q7. What is the difference between `revalidateTag` and `revalidatePath`?**

Answer: `revalidateTag("products")` clears ALL cached fetches tagged "products" across any page — precise, surgical. `revalidatePath("/products")` clears all data cached for that specific URL path. Use `revalidateTag` when same data appears on multiple pages. Use `revalidatePath` when you know exactly which page to refresh.

INTERMEDIATE**Q8. How do you cache a Prisma query in Next.js?**

Answer: Prisma queries are not `fetch()` — they have no built-in Next.js caching. Use `unstable_cache` to wrap the async Prisma function with a cache key, tags, and revalidate time. This gives it full ISR behaviour — cached on server, refreshed after N seconds, invalidated on demand via `revalidateTag`.

Expert / Syngenta Questions

EXPERT**Q9. Explain all 4 caching layers in Next.js App Router.**

Answer: 1. Request Memoization: in Lambda RAM, deduplicates `fetch()` per render — dies with Lambda. 2. Data Cache: Vercel persistent store, shared by all Lambdas, cleared by `revalidateTag`. 3. Full Route Cache: Vercel Edge CDN globally, complete HTML pages, Lambda not called for hits. 4. Router Cache: browser memory, RSC payloads for visited pages, 30s-5min, cleared by `router.refresh()`.

EXPERT**Q10. How does Suspense improve data fetching?**

Answer: Without Suspense: page waits for ALL data — slowest fetch blocks everyone. With Suspense: sections stream independently. Server sends available HTML immediately. Each Suspense boundary trickles in as its data resolves. Users see content progressively — slow weather widget does not block fast crop prices section.

SYNGENTA**Q11. Syngenta dashboard: static categories, hourly prices, real-time weather. How do you fetch each?**

Answer: Categories: `fetch(url)` — SSG, cached at build, served from CDN, Lambda never called. Crop prices: `next:{revalidate:3600}` — ISR, cached 1 hour, background refresh, fast CDN delivery. Weather: `cache:"no-store"` — SSR, fresh API call every request. Wrap each in Suspense so slow weather does not block fast sections.

EXPERT

Q12. Two fetches in same page have revalidate:3600 and revalidate:60. What happens?

Answer: Next.js uses the LOWEST revalidate for the entire page — so it revalidates every 60 seconds. Fix: move each fetch into its own async Server Component wrapped in a Suspense boundary. Each Suspense boundary has truly independent revalidation. The slow-changing data keeps its 1-hour cache, fast-changing data refreshes every minute.